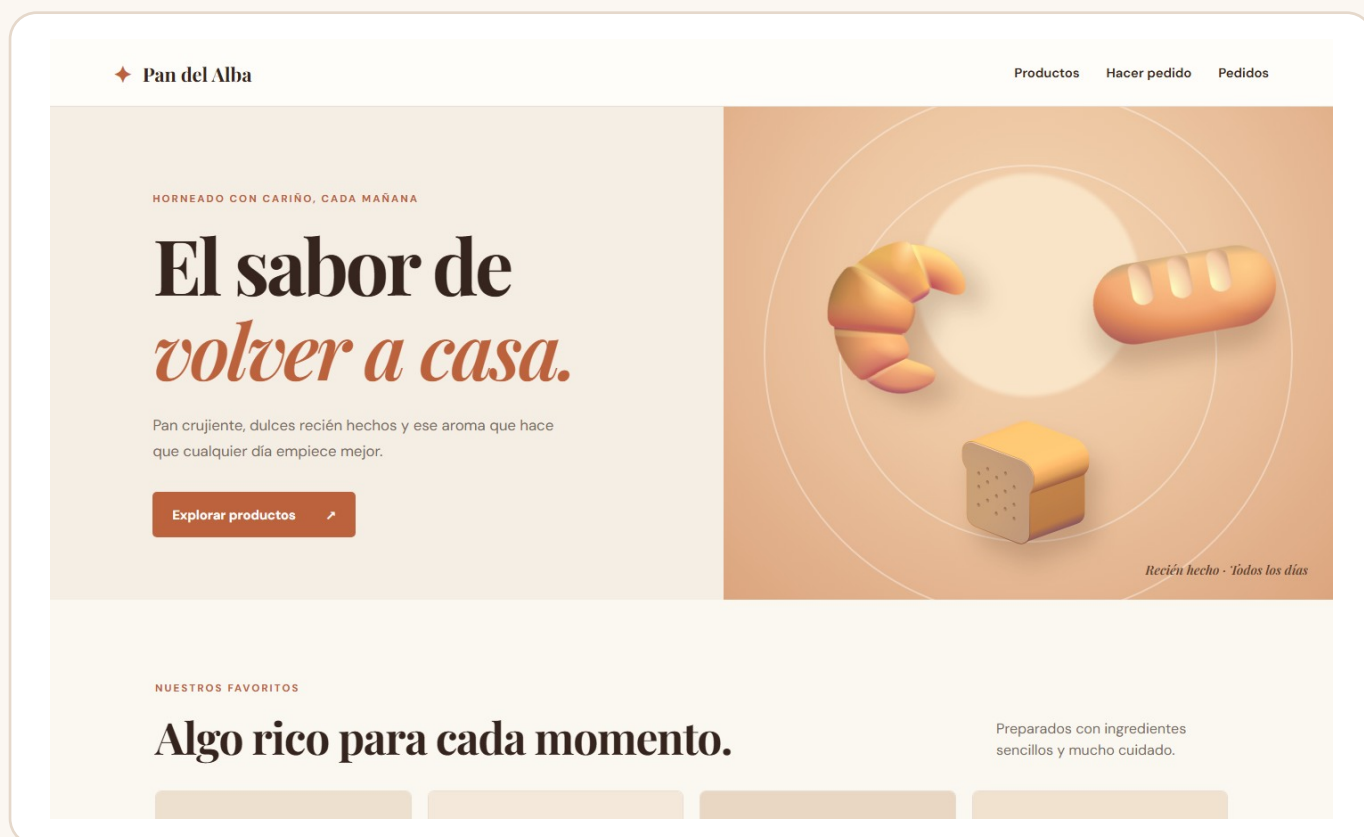


Pan del Alba

Práctica 2 · Flask + Jinja2 + SQLite + Docker

Una web de panadería con pedidos persistentes. Esta guía vincula cada prompt y cada paso técnico con una captura real del archivo o del resultado correspondiente.



Vista de la web final: Pan del Alba y su catálogo.

Qué demuestra la práctica

La imagen contiene la aplicación. El contenedor la ejecuta. El volumen `pedidos_data` conserva `panaderia.db` aunque se recree el contenedor.

Idea y estructura

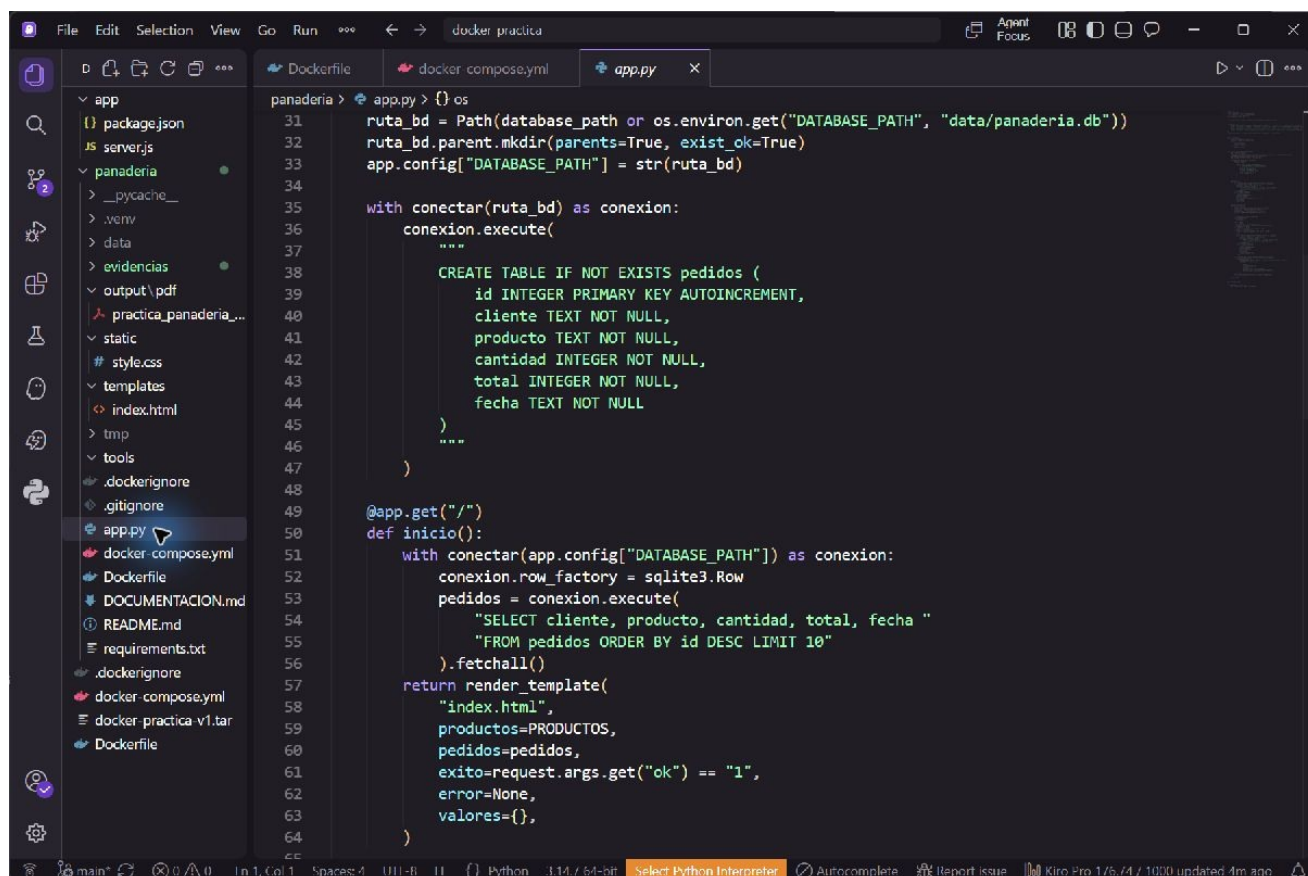
Prompt de trabajo propuesto para orientar este apartado.

PROMPT REDACTADO

Quiero hacer una práctica de Docker con una página web sencilla para una panadería. Propón una estructura de proyecto con Flask, plantillas Jinja2, CSS, SQLite, Dockerfile y Docker Compose. La web debe mostrar productos y permitir guardar pedidos.

RESULTADO VISIBLE

El proyecto separa app.py, la plantilla, los estilos y los archivos de Docker.



Captura real del editor: archivos del proyecto y código Flask/SQLite.

Diseño visual con Jinja2

Prompt de trabajo propuesto para orientar este apartado.

PROMPT REDACTADO

Diseña una página para una panadería llamada Pan del Alba usando HTML, CSS y plantillas Jinja2. Debe mostrar productos con precios, un formulario para registrar pedidos y una lista de pedidos guardados. Haz que se adapte a celular.

RESULTADO VISIBLE

La página muestra la identidad visual y el catálogo de productos.



Captura real de la web: portada y catálogo de Pan del Alba.

Flask y SQLite

Prompt de trabajo propuesto para orientar este apartado.

PROMPT REDACTADO

Crea una aplicación Flask que muestre productos, reciba un formulario de pedido y guarde nombre del cliente, producto, cantidad, total y fecha en SQLite. Valida los datos del formulario y muestra los últimos pedidos en la página.

RESULTADO VISIBLE

app.py crea la tabla pedidos y consulta los registros recientes.

```

panaderia > e app.py > () os
31 ruta_bd = Path(database_path or os.environ.get("DATABASE_PATH", "data/panaderia.db"))
32 ruta_bd.parent.mkdir(parents=True, exist_ok=True)
33 app.config["DATABASE_PATH"] = str(ruta_bd)
34
35 with conectar(ruta_bd) as conexion:
36     conexion.execute(
37         """
38         CREATE TABLE IF NOT EXISTS pedidos (
39             id INTEGER PRIMARY KEY AUTOINCREMENT,
40             cliente TEXT NOT NULL,
41             producto TEXT NOT NULL,
42             cantidad INTEGER NOT NULL,
43             total INTEGER NOT NULL,
44             fecha TEXT NOT NULL
45         )
46         """
47     )
48
49 @app.get("/")
50 def inicio():
51     with conectar(app.config["DATABASE_PATH"]) as conexion:
52         conexion.row_factory = sqlite3.Row
53         pedidos = conexion.execute(
54             "SELECT cliente, producto, cantidad, total, fecha "
55             "FROM pedidos ORDER BY id DESC LIMIT 10"
56         ).fetchall()
57     return render_template(
58         "index.html",
59         productos=PRODUCTOS,
60         pedidos=pedidos,
61         exito=request.args.get("ok") == "1",
62         error=None,
63         valores={},
64     )

```

Captura real de app.py: creación de la tabla SQLite y ruta inicial.

Dockerfile

Prompt de trabajo propuesto para orientar este apartado.

PROMPT REDACTADO

Escribe un Dockerfile para una aplicación Flask de Python. Debe instalar las dependencias de requirements.txt, copiar la aplicación, exponer el puerto 5000 y arrancar el servidor. La base SQLite estará en /app/data/panaderia.db.

RESULTADO VISIBLE

La receta de imagen usa Python 3.12, instala Flask y arranca app.py.

```
panaderia > Dockerfile
1  FROM python:3.12-slim
2
3  WORKDIR /app
4
5  COPY requirements.txt .
6  RUN pip install --no-cache-dir -r requirements.txt
7
8  COPY app.py .
9  COPY templates/ templates/
10 COPY static/ static/
11
12 ENV PYTHONDONTWRITEBYTECODE=1
13 ENV PYTHONUNBUFFERED=1
14 ENV DATABASE_PATH=/app/data/panaderia.db
15
16 EXPOSE 5000
17
18 CMD ["python", "app.py"]
19
```

Captura real del Dockerfile del proyecto de panadería.

Compose y volumen

Prompt de trabajo propuesto para orientar este apartado.

PROMPT REDACTADO

Escribe un docker-compose.yml para la web Flask. Publica el puerto 5000 y monta un volumen con nombre en /app/data, de modo que panaderia.db permanezca aunque se elimine y recree el contenedor.

RESULTADO VISIBLE

Compose define el servicio web y el volumen pedidos_data.

```
panaderia > docker-compose.yml
1  services:
2    web:
3      build: .
4      ports:
5        - "5000:5000"
6      environment:
7        DATABASE_PATH: /app/data/panaderia.db
8      volumes:
9        - pedidos_data:/app/data
10
11 volumes:
12   pedidos_data:
13
```

Captura real de Compose: puerto 5000 y volumen /app/data.

Comprobación

Prompt de trabajo propuesto para orientar este apartado.

PROMPT REDACTADO

Explica cómo probar la página, registrar un pedido y demostrar que sigue guardado después de ejecutar `docker compose down` y `docker compose up -d`. Incluye los comandos para ver los contenedores, registros y volúmenes.

RESULTADO VISIBLE

El pedido de prueba continúa visible tras recrear el contenedor.

Esta es una demostración para la práctica. Al enviar el formulario, tu pedido se guarda en SQLite.

Los pedidos permanecen guardados gracias al volumen de Docker.

Selecciona un producto ▼

Cantidad

1

Guardar pedido ➤

DATOS GUARDADOS

Pedidos recientes

Estos registros se leen desde la base de datos SQLite.

Pedido de prueba Marraqueta · 3 unidades	Bs 6 25/09/2026 11:50
--	---------------------------------

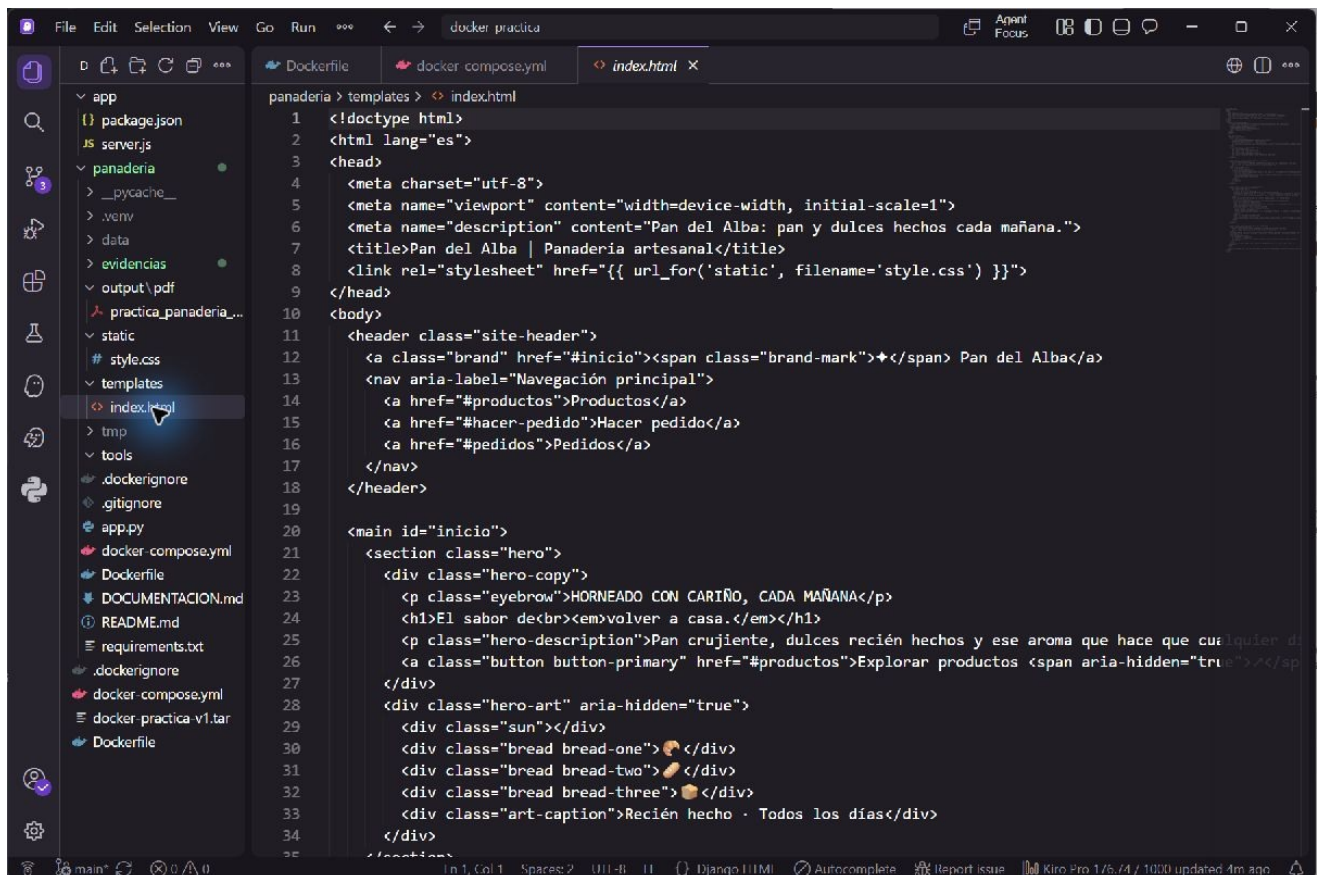
Captura real tras la recreación: pedido conservado en la web.

Preparar la aplicación

Se organizan Flask, Jinja2, CSS y SQLite antes de construir la imagen.

ARCHIVO O COMANDOS

Archivos: app.py, requirements.txt, templates/index.html y static/style.css.



Captura del editor: plantilla HTML/Jinja2 y estructura de carpetas.

COMPROBACIÓN

La aplicación y sus dependencias existen en la carpeta panaderia.

Escribir el Dockerfile

El Dockerfile especifica la base Python, las dependencias, el código y el arranque en el puerto 5000.

ARCHIVO O COMANDOS

FROM python:3.12-slim → COPY → RUN pip install → EXPOSE 5000 → CMD

```
panaderia > Dockerfile
1  FROM python:3.12-slim
2
3  WORKDIR /app
4
5  COPY requirements.txt .
6  RUN pip install --no-cache-dir -r requirements.txt
7
8  COPY app.py .
9  COPY templates/ templates/
10 COPY static/ static/
11
12 ENV PYTHONDONTWRITEBYTECODE=1
13 ENV PYTHONUNBUFFERED=1
14 ENV DATABASE_PATH=/app/data/panaderia.db
15
16 EXPOSE 5000
17
18 CMD ["python", "app.py"]
19
```

Captura del Dockerfile que se utilizó para construir la imagen.

COMPROBACIÓN

La base SQLite se ubica en /app/data/panaderia.db.

Configurar Compose

Compose publica el puerto 5000 y conecta /app/data con un volumen con nombre.

ARCHIVO O COMANDOS

docker compose config

```
panaderia > 📄 docker-compose.yml
1  services:
2    web:
3      build: .
4      ports:
5        - "5000:5000"
6      environment:
7        DATABASE_PATH: /app/data/panaderia.db
8      volumes:
9        - pedidos_data:/app/data
10
11 volumes:
12   pedidos_data:
13
```

Captura de docker-compose.yml: servicio web y volumen pedidos_data.

COMPROBACIÓN

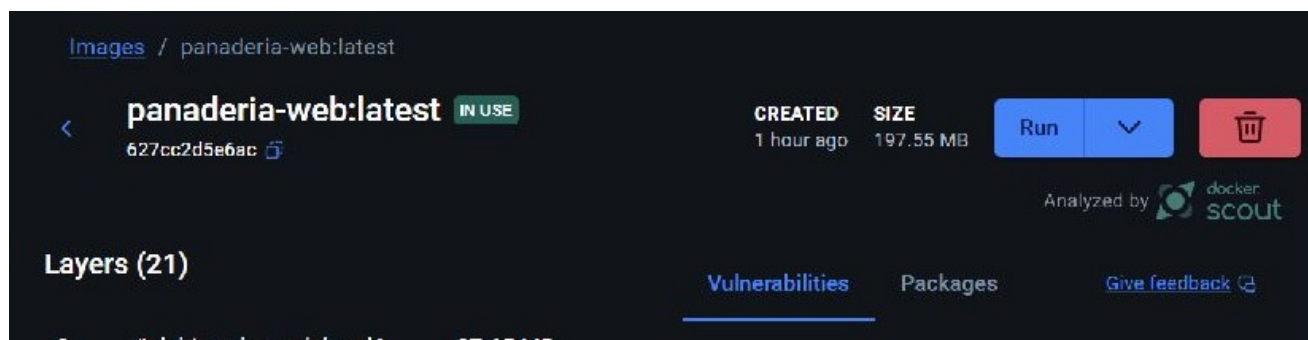
La configuración muestra web, 5000:5000 y pedidos_data:/app/data.

Construir la imagen

Docker lee el Dockerfile y crea la imagen local panaderia-web. Todavía no se necesita un contenedor.

ARCHIVO O COMANDOS

```
docker compose build
docker image ls panaderia-web
```



Ficha de la imagen: panaderia-web:latest, en uso.

1.6 GB / 2.32 GB in use 6 images Last refresh: 58 minutes ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☐	mysql	latest	ade067ae2fb1	4 days ago	1.29 GB	
☐	docker-pract	latest	19ccfba50e43	7 days ago	193.26 MB	
☐	docker-practi	v1	84210026e98b	7 days ago	193.26 MB	
☐	postgres	16-alpine	cf78e76683b9	1 month ago	419.68 MB	
☐	postgres	17-alpine	18cfe3ef5e68	1 month ago	423.79 MB	
☐	panaderia-wi	latest	627cc2d5e6ac	52 minutes ag	197.55 MB	

Imagen visible en la lista local de Docker Desktop.

COMPROBACIÓN

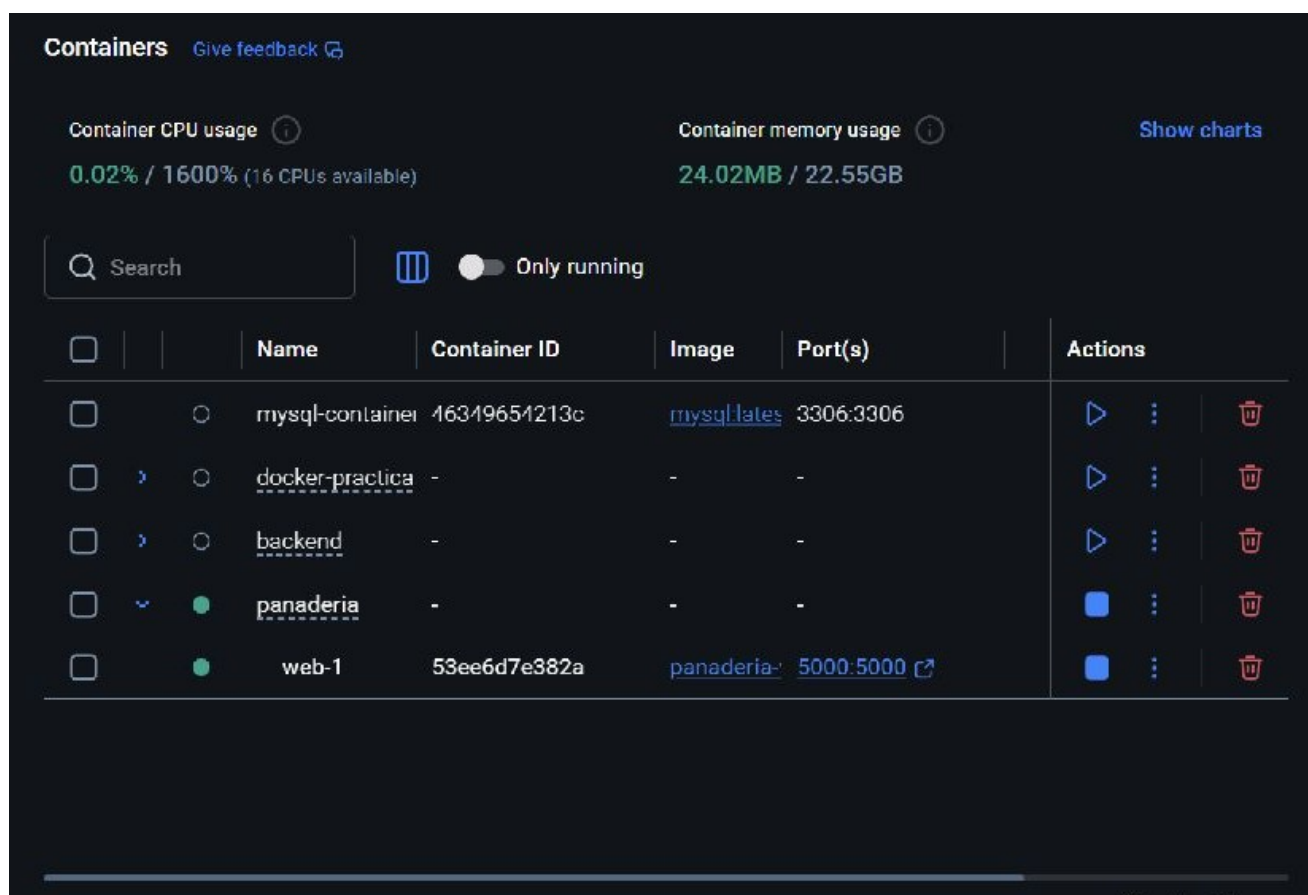
La imagen aparece con su etiqueta latest y un ID de imagen.

Crear el contenedor

Se inicia el servicio en segundo plano y se comprueba su estado y puerto.

ARCHIVO O COMANDOS

```
docker compose up -d
docker compose ps
docker compose logs web
```



Captura de Docker Desktop: web-1 en ejecución y puerto 5000:5000.

COMPROBACIÓN

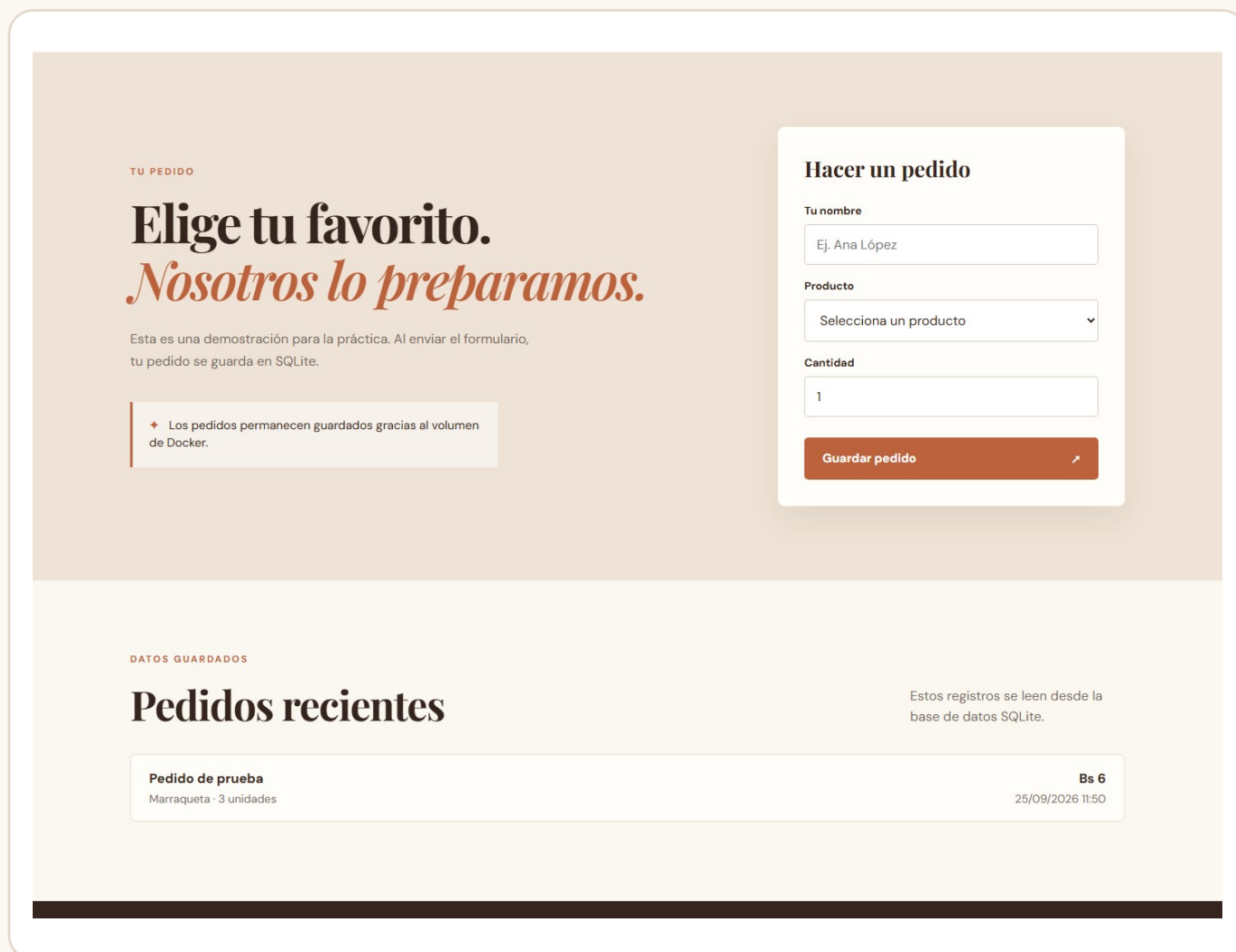
El contenedor figura activo y Flask responde por el puerto 5000.

Probar un pedido

En localhost:5000 se navega el catálogo y se envía el formulario de pedido.

ARCHIVO O COMANDOS

Abrir <http://localhost:5000> y registrar 3 marraquetas para Pedido de prueba.



The screenshot displays a web application interface. On the left, a section titled 'TU PEDIDO' features the heading 'Elige tu favorito. Nosotros lo preparamos.' and a note: 'Esta es una demostración para la práctica. Al enviar el formulario, tu pedido se guarda en SQLite.' Below this is a tip: 'Los pedidos permanecen guardados gracias al volumen de Docker.' On the right, a 'Hacer un pedido' form is shown with fields for 'Tu nombre' (filled with 'Ej. Ana López'), 'Producto' (a dropdown menu with 'Selecciona un producto'), and 'Cantidad' (filled with '1'). A 'Guardar pedido' button is at the bottom of the form. Below the form, a 'DATOS GUARDADOS' section titled 'Pedidos recientes' shows a table with one entry: 'Pedido de prueba' (Marraqueta - 3 unidades) with a total of 'Bs 6' and a timestamp '25/09/2026 11:50'. A note states: 'Estos registros se leen desde la base de datos SQLite.'

Captura de la web: formulario y pedido guardado, con total Bs 6.

COMPROBACIÓN

El pedido aparece en Pedidos recientes.

Demostrar persistencia

El contenedor se elimina y recrea, pero el volumen conserva el archivo SQLite.

ARCHIVO O COMANDOS

```
docker compose down
docker compose up -d
docker compose ps
```

☐		Name	Container ID	Image	Port(s)	Actions
☐	☐	mysql-container	46349654213c	mysql:latest	3306:3306	
☐		docker-practica	-	-	-	
☐		backend	-	-	-	
☐		panaderia	-	-	-	
☐	☒	web-1	53ee6d7e382a	panaderia:	5000:5000	

Contenedor web-1 recreado y activo en Docker Desktop.

DATOS GUARDADOS

Pedidos recientes

Pedido de prueba

Marraqueta · 3 unidades

Bs 6

25/09/2026 11:50

Estos registros se leen desde la base de datos SQLite.

El pedido sigue guardado en la página después de recrearlo.

COMPROBACIÓN

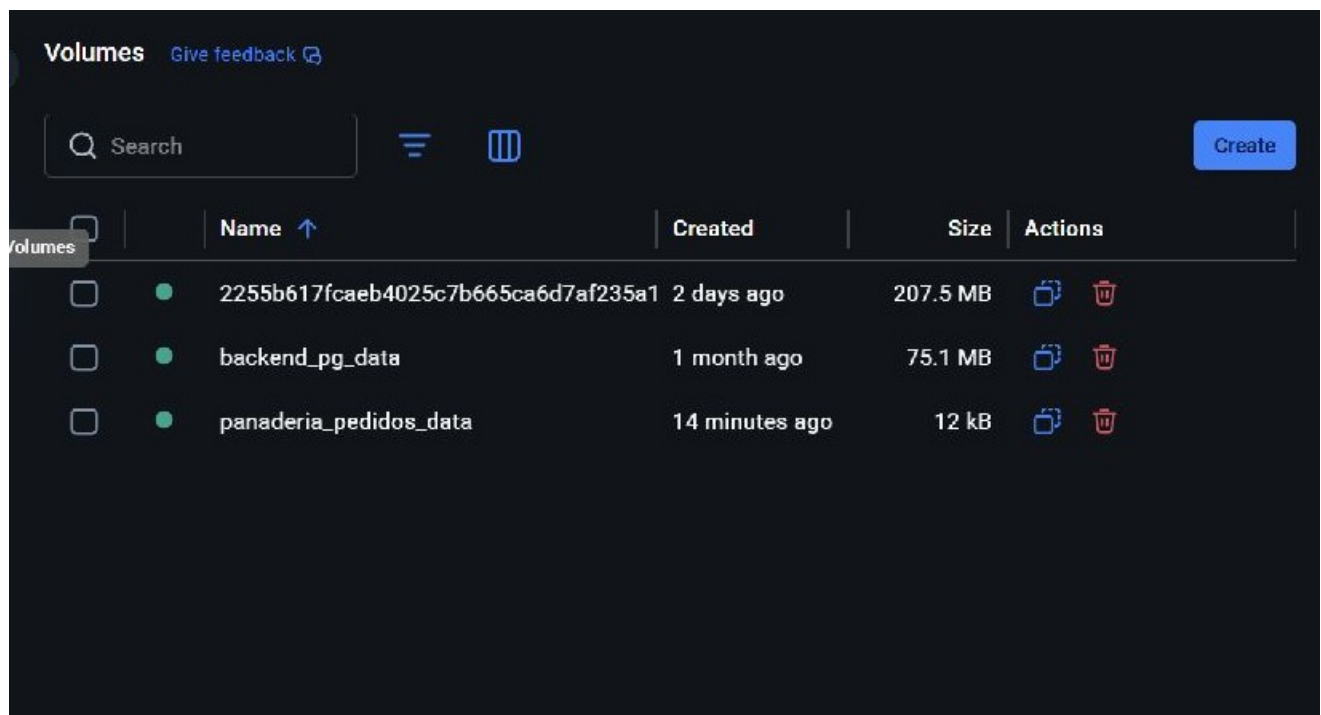
El ID del contenedor cambió y el pedido continuó visible.

Verificar el volumen

El volumen permite guardar la base de datos fuera del ciclo de vida del contenedor.

ARCHIVO O COMANDOS

```
docker volume ls
docker volume inspect panaderia_pedidos_data
```



The screenshot shows the Docker Desktop interface with the 'Volumes' tab selected. It displays a table of existing volumes. The table has columns for Name, Created, Size, and Actions. There are three volumes listed: a long ID, 'backend_pg_data', and 'panaderia_pedidos_data'. The 'panaderia_pedidos_data' volume is highlighted, indicating it is the one being used in the current step.

Name	Created	Size	Actions
2255b617fcaeb4025c7b665ca6d7af235a1	2 days ago	207.5 MB	 
backend_pg_data	1 month ago	75.1 MB	 
panaderia_pedidos_data	14 minutes ago	12 kB	 

Captura de Docker Desktop: volumen panaderia_pedidos_data en uso.

COMPROBACIÓN

No ejecutar docker compose down --volumes durante esta prueba.

Imagen y contenedor

Reproducción real por comandos, adaptada de la práctica 2: Flask usa el puerto 5000 y la prueba directa publica 5001 para no afectar el servicio principal.

COMANDOS EJECUTADOS

```
docker build -t panaderia:practica2 .
docker run -d --name panaderia-cli -p 5001:5000 -v panaderia_cli_datos:/app/data panaderia:practica2
```

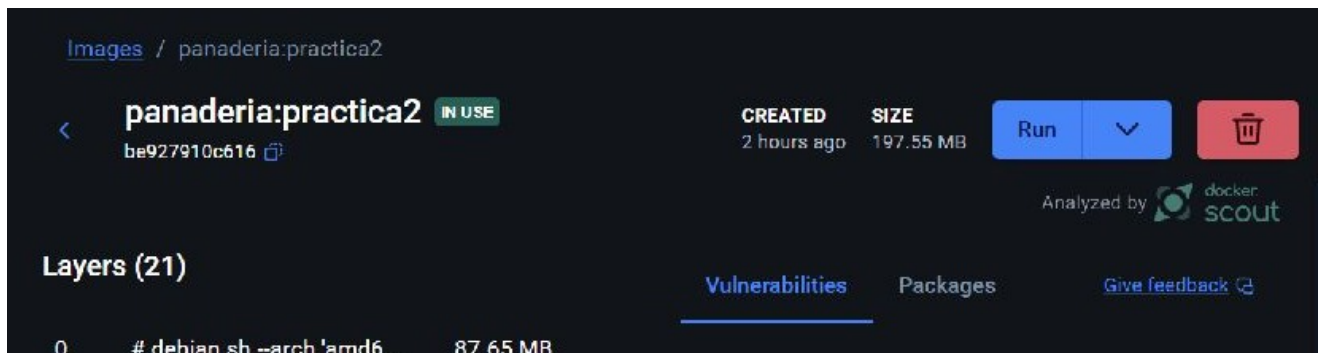


Imagen panaderia:practica2 creada por docker build.

☐		Name	Container ID	Image	Port(s)	Actions
☐	☐	mysql-container	46349654213c	mysql:latest	3306:3306	<input type="play"/> <input type="vertical-ellipsis"/> <input type="trash"/>
☐	☒	panaderia-cli	a13ddc5ec171	panaderia:practica2	5001:5000 <input type="external-link"/>	<input type="play"/> <input type="vertical-ellipsis"/> <input type="trash"/>
☐	☒	panaderia-sond	50b5f3b8cbc3	panaderia:practica2		<input type="play"/> <input type="vertical-ellipsis"/> <input type="trash"/>
☐	☒	docker-practica	-	-	-	<input type="play"/> <input type="vertical-ellipsis"/> <input type="trash"/>
☐	☒	backend	-	-	-	<input type="play"/> <input type="vertical-ellipsis"/> <input type="trash"/>
☐	☒	panaderia	-	-	-	<input type="play"/> <input type="vertical-ellipsis"/> <input type="trash"/>
☐	☒	web-1	53ee6d7e382a	panaderia:practica2	5000:5000 <input type="external-link"/>	<input type="play"/> <input type="vertical-ellipsis"/> <input type="trash"/>

panaderia-cli funciona en 5001; el servicio Compose sigue en 5000.

Comprobación real: GET <http://localhost:5001> devolvió HTTP 200.

Montaje, red y Compose

Se probaron el bind mount y la comunicación entre dos contenedores sin cambiar el código de la panadería ni sustituir su Compose principal.

PASO 7 · BIND MOUNT

Se montó static/ en /app/static como solo lectura. La comprobación dentro de un contenedor temporal devolvió: BIND_MOUNT_OK True. No se afirmó haber probado recarga en caliente.

		Name	Container ID	Image	Port(s)	Actions
☐	☐	mysql-container	46349654213c	mysql:latest	3306:3306	
☐	☒	panaderia-cli	a13ddc5ec171	panaderia:	5001:5000	
☐	☒	panaderia-sond	50b5f3b8cbc3	panaderia:		
☐	☐	docker-practica	-	-	-	
☐	☐	backend	-	-	-	
☐	☒	panaderia	-	-	-	
☐	☒	web-1	53ee6d7e382a	panaderia:	5000:5000	

Contenedores web, CLI y sonda visibles en Docker Desktop.

PASO 8 · RED PERSONALIZADA

docker network create panaderia-lab; docker network connect panaderia-lab panaderia-cli. Desde panaderia-sonda se consultó http://panaderia-cli:5000 y se obtuvo RED_OK_HTTP 200.

PASO 9 · COMPOSE ADAPTADO

docker compose ps mostró panaderia-web-1 activo en 5000:5000. El Compose principal tiene un servicio; no se presenta como multiservicio.

Evidencias y repositorio

Las capturas anteriores muestran artefactos y resultados reales; no representan pantallas de una conversación con IA. Los prompts son propuestas de trabajo redactadas para documentar la planificación.

Historial construido por etapas

- feat: crear web de panaderia con Flask y SQLite
- build: contenerizar la app y persistir pedidos en volumen
- docs: explicar arquitectura, prompts y prueba de volumen
- docs: añadir evidencias reales y PDF de entrega
- docs: documentar creacion de imagen y contenedor paso a paso
- docs: incorporar capturas a planificacion y aplicacion

Repositorio público

<https://github.com/Chucharizard/pan-del-alba-docker>

Resultado: un pedido de 3 marraquetas (Bs 6) siguió visible después de docker compose down y docker compose up -d. El contenedor cambió de ID; el volumen panaderia_pedidos_data permaneció.

Construccion de la imagen

Captura autentica: despues del primer intento desde system32, se entro al directorio del proyecto y el build finalizo.

```
PS C:\WINDOWS\system32> docker run --rm --mount "type=bind,source=$static,target=/app/static,readonly" panaderia:practica2 python -c "from pathlib import Path; print('BIND_MOUNT_OK', Path('/app/static/style.css').is_file())"
invalid argument "type=bind,source=,target=/app/static,readonly" for "--mount" flag: invalid value for 'source': value is empty

Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Run 'docker run --help' for more information
PS C:\WINDOWS\system32> docker exec panaderia-sonda python -c "import urllib.request; print('RED_OK_HTTP', urllib.request.urlopen('http://panaderia-cli:5000').status)"
RED_OK_HTTP 200
PS C:\WINDOWS\system32> cd "C:\Users\willy\Downloads\OneDrive_1_18-9-2026\docker-practica\docker-practica\panaderia"
PS C:\Users\willy\Downloads\OneDrive_1_18-9-2026\docker-practica\docker-practica\panaderia> docker build -t panaderia:practica2 .
[+] Building 1.6s (13/13) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 348B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 1.3s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 85B                                       0.0s
=> [1/7] FROM docker.io/library/python:3.12-slim@sha256:2f17fc044b579bab302c2e8054d3a686e2cb9a83de48e70534b94cd8 0.0s
=> => resolve docker.io/library/python:3.12-slim@sha256:2f17fc044b579bab302c2e8054d3a686e2cb9a83de48e70534b94cd8 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 194B                                       0.0s
=> CACHED [2/7] WORKDIR /app                                       0.0s
=> CACHED [3/7] COPY requirements.txt .                             0.0s
=> CACHED [4/7] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/7] COPY app.py .                                       0.0s
=> CACHED [6/7] COPY templates/ templates/                         0.0s
=> CACHED [7/7] COPY static/ static/                               0.0s
=> exporting to image                                              0.1s
=> => exporting layers                                                0.0s
=> => exporting manifest sha256:e8cda5c9cf42d53426f9b58011f6aad930361942962b415fd9ba03c6ec388e2c 0.0s
=> => exporting config sha256:0ca0ee103be28f42180f332ee6c33393e86192096a0b0409131a42c746e66293 0.0s
=> => exporting attestation manifest sha256:105fdbdf56c5039e606b9bec65638a091618506bf5bbb72ff65865579de37295 0.0s
=> => exporting manifest list sha256:76ef8958ccf7f16d157a092a36bdae36cbc1d6579a2ff28b714f67eb394f9b3f 0.0s
=> => naming to docker.io/library/panaderia:practica2              0.0s
=> => unpacking to docker.io/library/panaderia:practica2           0.0s
PS C:\Users\willy\Downloads\OneDrive_1_18-9-2026\docker-practica\docker-practica\panaderia> |
```

Se ve el contexto correcto de Dockerfile y la imagen panaderia:practica2 etiquetada.

Nota

El error inicial fue por ejecutar desde la carpeta equivocada; se corrigio cambiando a la carpeta panaderia.

Comprobacion del bind mount

Captura autentica de PowerShell con la ruta local definida antes de ejecutar docker run.

```
=> => exporting config sha256:0ca0ee105be207421001932ee0c9999e00192090a000409191a42c740e00299 0.0s
=> => exporting attestation manifest sha256:105fdbdf56c5039e606b9bec65638a091618506bf5bbb72ff65865579de37295 0.0s
=> => exporting manifest list sha256:76ef8958ccf7f16d157a092a36bdae36cbc1d6579a2ff28b714f67eb394f9b3f 0.0s
=> => naming to docker.io/library/panaderia:practica2 0.0s
=> => unpacking to docker.io/library/panaderia:practica2 0.0s
PS C:\Users\willy\Downloads\OneDrive_1_18-9-2026\docker-practica\docker-practica\panaderia> $static = (Resolve-Path .\static).Path
PS C:\Users\willy\Downloads\OneDrive_1_18-9-2026\docker-practica\docker-practica\panaderia> docker run --rm --mount "type=bind,source=$static,target=/app/static,readonly" panaderia:practica2 py
thon -c "from pathlib import Path; print('BIND_MOUNT_OK', Path('/app/static/style.css').is_file())"
BIND_MOUNT_OK True
PS C:\Users\willy\Downloads\OneDrive_1_18-9-2026\docker-practica\docker-practica\panaderia> |
```

La salida BIND_MOUNT_OK True confirma que el contenedor ve el archivo local static/style.css.

Nota

La guia tambien solicita cambiar el codigo para mostrar el nombre completo y reiniciar el contenedor. Esta evidencia comprueba el montaje, pero no ese cambio de respuesta.

Comunicacion por la red Docker

Captura autentica de la peticion HTTP desde panaderia-sonda hacia panaderia-cli.

```
36 Run 'docker run --help' for more information
PS C:\WINDOWS\system32> docker exec panaderia-sonda python -c "import urllib.request; print('RED_OK_HTTP', urllib.reques
N t.urlopen('http://panaderia-cli:5000').status)"
RED_OK_HTTP 200
PS C:\WINDOWS\system32> |
```

La salida RED_OK_HTTP 200 confirma respuesta del servicio por nombre dentro de la red Docker.

Nota

La aplicacion Compose principal sigue publicada en el puerto 5000; el contenedor de prueba utiliza el 5001.